CMPT 295

Unit - Machine-Level Programming

Lecture 12 – Assembly language basics:
memory addressing modes,
`leaq` instruction and
arithmetic & logical operations

Practice and DEMO

Last Lecture

- x86-64 instructions can have various types of **operands**
  1. Immediate (constant integral value)
  2. Register (16 registers)
  3. Memory addressing mode (various forms)
     - General Syntax of operand: **`Imm(`r_b`, `r_i`, s)`**
- **`leaq`** - load effective address instruction
- Arithmetic & logical instructions
  - Arithmetic instructions: **`add*, sub*, imul* inc*, dec*, neg*, not*`**
  - Logical instructions: **`and*, or*, xor*`**
  - Shift instructions: **`sal*, sar*, shr*`**

1. **Absolute**
2. **Indirect**
3. **"Base + displacement"**
4. **2 indexed**
5. **4 scaled indexed**

* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

Today's Menu

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Practice and DEMO!

Demo

1. `makefile`
   - when we compile our own `*.s` files with `*.c` files

   versus
   - when we compile only `*.c` files
2. `gcc` uses `leaq` for addition -> `sum_store.c`
3. Writing our own assembly code (`arith.s`) using arithmetic instructions of x86-64 assembly language
4. How would `gcc` compile our `arith.c` into `arith.s`?
5. Disassembler: `objdump -d ar > ar.objdump`

Demo

arith.c

```c
long arith(long x, long y, long z) {
  long t1 = x + y;
  long t2 = z + t1;
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

- Observation: C compiler will figure out different instruction combinations to carry out the computations in our C code

arith_mine.s

```
    .globl  arith
arith:  # mine
    # x -> %rdi, y -> %rsi, z -> %rdx
    # addq %rsi, %rdi # Bad! because overwrites %rdi which is needed later on for "t3 = x + 4"
    # addq %rdi, %rsi # Bad! because overwrites %rsi which is needed later on for "t4 = y * 48"
    leaq (%rdi, %rsi), %rax  # t1 -> %rax <- function's returned value
    #OR      movq %rdi, %rax # x -> %rax
    #        addq %rsi, %rax # t1 = x + y; t1 -> %rax
    addq %rdx, %rax          # t2 = z + t1; t2 -> %rax
    addq $4, %rdi            # t3 = x + 4; t3 -> %rdi
    imulq $48, %rsi          # t4 = y * 48; t4 -> %rsi
    addq %rsi, %rdi          # t5 = t3 + t4; t5 -> %rdi
    imulq %rdi, %rax         # rval = t2 * t5; rval -> %rax
    ret
```

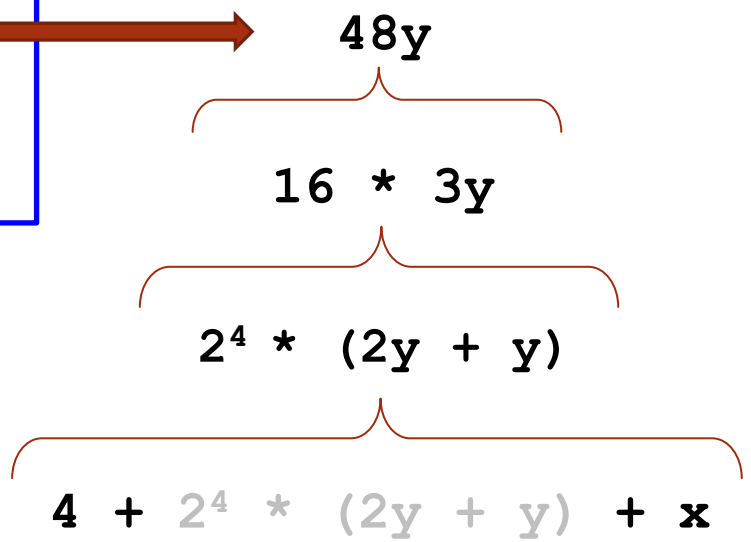
arith_gcc.s

```
arith:
.LFB0:
    .cfi_startproc
    endbr64
    movq    %rdx, %rax
    leaq    (%rsi,%rsi,2), %rcx
    salq    $4, %rcx
    leaq    4(%rdi,%rcx), %rdx
    addq    %rsi, %rdi
    addq    %rdi, %rax
    imulq   %rdx, %rax
    ret
```

Demo

arith.c

```c
long arith(long x, long y, long z) {
  long t1 = x + y;
  long t2 = z + t1;
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```



arith_gcc.s

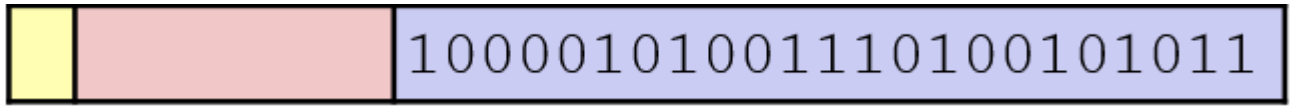
```
arith:
.LFB0:
    .cfi_startproc
    endbr64
    movq    %rdx, %rax
    leaq    (%rsi,%rsi,2), %rcx
    salq    $4, %rcx
    leaq    4(%rdi,%rcx), %rdx
    addq    %rsi, %rdi
    addq    %rdi, %rax
    imulq   %rdx, %rax
    ret
```

Summary

- Demo
  - Observation: C compiler will figure out different instruction combinations to carry out the computations in our C code

Review Questions

1. Rounding M = 1.10000101001110100101011101 would produce the following frac: True or False



2. These C statements print **1** (true) on the screen as the result of executing **aFloat == anInt**: True or False

```
float aFloat = 12345; int anInt = aFloat;
printf("%.2f == %d -> %d", aFloat, anInt, aFloat == anInt);
```

3. %rcx would contain **5** once the following instruction has executed True or False

```
movq 0x10(%rbx , %rax , 4), %rcx
```

if we have:

Registers	Value
%rax	0x0100
%rbx	0x0200
%rcx	128

Address	Value
0x0600	5
0x0400	256
0x0610	8

Next lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations